SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

삼성청년 SW·AI아카데미

DataBase

SAMSUNG SW AI ACADEMY FOR YOUTH

목차

1. 내장 함수 - 집계 함수
2. 내장 함수 - 집계 함수 응용
3. DML

SAMSUNG SW AI ACADEMY FOR YOUTH

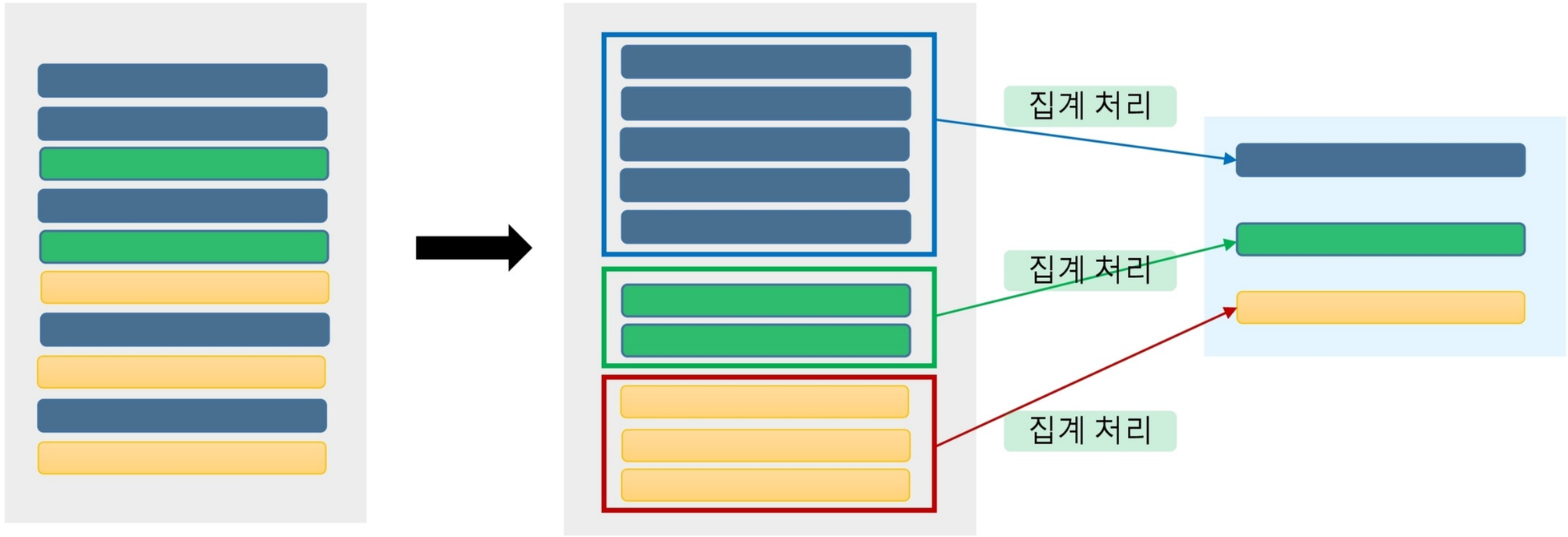
함께가요 미래로!
Enabling People

내장 함수 - 집계 함수

내장 함수 - 집계 함수

집계 함수

- 결과를 만족하는 전체 행을 하나 이상의 컬럼을 기준으로 그룹화하여 그룹별로 결과를 반환하는 함수
- 통계 결과를 제공



내장 함수 - 집계 함수

집계 함수의 종류

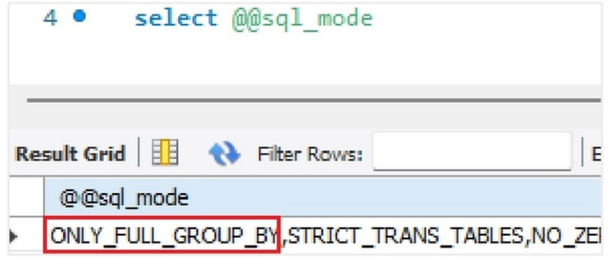
- NULL값을 집계하지 않음에 주의
  - COUNT(*) 제외

함수	설명
COUNT([*ALL*] 컬럼명) COUNT([DISTINCT] 컬럼명 [,컬럼명,...])	그룹 내 레코드의 해당 컬럼이 NULL 값이 아닌 레코드 수 COUNT(*) : 전체 레코드 수
SUM([DISTINCT | *ALL*] 컬럼명)	그룹 내 레코드의 해당 컬럼 값의 합계
AVG([DISTINCT | *ALL*] 컬럼명)	그룹 내 레코드의 해당 컬럼 값의 평균
MAX([DISTINCT | *ALL*] 컬럼명)	그룹 내 레코드의 해당 컬럼 값 중 최대값
MIN([DISTINCT | *ALL*] 컬럼명)	그룹 내 레코드의 해당 컬럼 값 중 최소값

내장 함수 - 집계 함수

GROUP BY절

```
SELECT      . . .
FROM        . . .
WHERE       . . .
GROUP BY  컬럼명 | 표현식 [, 컬럼명 | 표현식, … ]
. . .
```

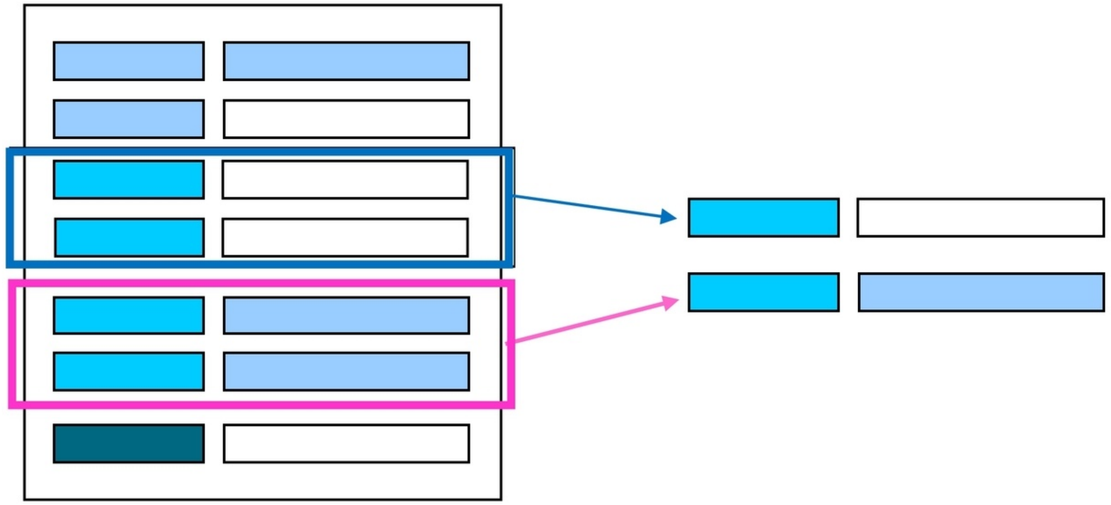


- 결과 집합의 하위 데이터 그룹을 만듦
- GROUP BY절에 기술한 컬럼이나 표현식을 기준으로 결과 집합 전체 행을 그룹별로 나눔
- 각 그룹별로 SELECT절에 기술한 집계 함수가 적용
- SELECT절의 모든 요소는 GROUP BY 절의 표현식, 집계 함수를 포함하는 표현식 또는 상수만 가능 **ONLY_FULL_GROUP_BY 모드**
- SELECT절에 기술한 컬럼 중, 집계 함수에 사용되지 않은 컬럼은 GROUP BY절에 반드시 기술되어야 함 **ONLY_FULL_GROUP_BY 모드**
- WHERE절에는 집계 함수를 사용할 수 없음
    - Grouping 이전에 WHERE 절을 사용하여 그룹 대상 집합을 먼저 선택

내장 함수 - 집계 함수

다중 컬럼을 이용한 그룹화

- 하나 이상의 컬럼을 사용하여 그룹화
- 기준 1로 그룹을 나누고 다음 기준(기준2)을 적용하여 기준 1로 형성된 그룹별로 다시 서브 그룹을 나눔



원본 데이터

지역	제품	매출액
서울	A	100
서울	A	120
서울	B	200
서울	B	180
서울	C	150
부산	A	300
부산	A	260
부산	B	120
부산	C	180
대구	A	220
대구	B	160
대구	C	140

다중 컬럼 그룹화 결과 (지역 → 제품)

서울	제품 A	서울	A	100
		서울	A	120
	제품 B	서울	B	200
		서울	B	180
	제품 C	서울	C	150
부산	제품 A	부산	A	300
		부산	A	260
	제품 B	부산	B	120
	제품 C	부산	C	180
대구	제품 A	대구	A	220
	제품 B	대구	B	160
	제품 C	대구	C	140

지역: 서울 (기준1)으로 그룹화 → 제품(기준2)로 다시 그룹화

지역: 부산 (기준1)으로 그룹화 → 제품(기준2)로 다시 그룹화

지역: 대구 (기준1)으로 그룹화 → 제품(기준2)로 다시 그룹화

내장 함수 - 집계 함수

HAVING절

```
SELECT      . . .
FROM        . . .
WHERE       . . .
GROUP BY    컬럼명 | 표현식 [, 컬럼명 | 표현식, … ]
HAVING      조건
. . .
```

- GROUP BY 절에 의해 생성된 그룹을 대상으로 조건을 적용하여 그룹을 Filtering하기 위해 사용
- 집계 관련 조건절

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

내장 함수 - 집계 함수 응용

내장 함수 - 집계 함수

WITH ROLLUP

```
SELECT      . . .
FROM        . . .
WHERE       . . .
GROUP BY    컬럼명 | 표현식 [, 컬럼명 | 표현식, ... ] WITH ROLLUP
HAVING      조건
. . .
```

- 그룹 항목 총계나 각 그룹별 소계가 필요한 경우 사용

부서	월 급여합
10	8750
20	14875
30	11400
	35025

부서	직무	월 급여합
10	CLERK	1300
10	MANAGER	2450
10	PRESIEDENT	5000
10		**8750**
20	...	...
		35025

내장 함수 - 집계 함수

GROUPING함수

- 조회되는 컬럼의 NULL값이 실제 NULL값인지 ROLLUP에 의해 생성된 NULL값인지 판단
- 실제 NULL값 또는 NULL값이 아닌 값
  - 0 리턴
- ROLLUP에 의한 NULL값
  - 1 리턴

상사가 없어서 NUL.L

ROLLUP처리에 의한 NULL

부서	상사 사번	월 급여합
10	NULL	5000
10	7782	1300
10	7839	2450
10	**NULL**	**8750**
20	…	…
		35025

부서	상사 사번	월 급여합
10	**없음**	5000
10	7782	1300
10	7839	2450
10	**관리자 소계**	**8750**
20	…	…
총계	**총계**	**35025**

내장 함수 - 집계 함수

피봇 테이블(Pivot Table)

- 데이터를 **행**, **열**, **값**으로 재구성하여 정보를 쉽게 파악할 수 있음
- 행 및 열 기준으로 데이터 그룹화
  - 특정 기준(예: 직무, 입사월 등)에 따라 데이터를 그룹화하여 분석 가능
- 값 요약
  - 데이터를 합계, 평균, 최대값, 최소값 등으로 요약하여 결과 조회 가능

부서별 직무별 사원수

부서	CLERK	MANAGER	PRESIDENT	ANALYST	SALESMAN
10	1	1	1	0	0
20	2	1	0	2	0
30	1	1	0	0	4

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

DML

DML

DML

- Data Manipulation Language
- 테이블의 데이터를 조작하기 위한 명령어

종류	의 미
INSERT	테이블에 1 개 또는 여러 개의 새로운 데이터 삽입
UPDATE	테이블에 존재하는 1 개 또는 여러 개의 기존 데이터 수정
DELETE	테이블에 존재하는 1 개 또는 여러 개의 데이터 삭제

DML

INSERT

- 새로운 행을 삽입
- 레코드 수 증가
- 단일 행 삽입

```
INSERT INTO 테이블명 [(컬럼명 [, 컬럼명...])]
VALUES ({값| Sub Query } [, 값...]);
```

- INTO절 컬럼 리스트에 명시한 컬럼에 맞게 VALUES 절에서 대응되는 컬럼 값을 입력
- INTO절 컬럼 리스트를 명시하지 않으면 테이블 생성시 정의한 모든 컬럼을 생성 순서와 동일한 순서로 입력

INSERT

- 다중 행 삽입

```
INSERT INTO 테이블명 [(컬럼명 [, 컬럼명...])]
VALUES (값 [, 값...]),
       (값 [, 값...]),
        ....
       (값 [, 값...]);
```

생략 가능한 컬럼

1. NULL이 허용된 컬럼
2. DEFAULT가 설정된 컬럼
3. AUTO_INCREMENT가 설정된 컬럼

```
INSERT INTO 테이블명 [(컬럼명 [, 컬럼명...])]
Sub Query
```

- INSERT 명령문의 VALUES절 대신 서브 쿼리에서 검색된 결과 집합을 한꺼번에 입력
- 서브 쿼리의 결과 집합은 INSERT 명령문에 지정된 칼럼 개수와 데이터 타입이 일치해야 함

DML

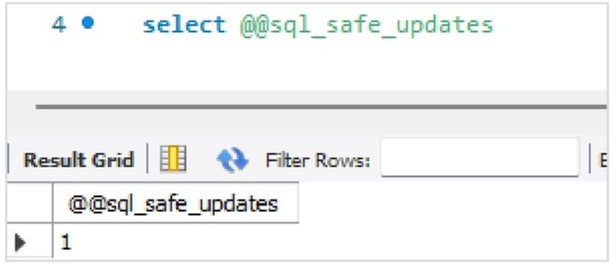
- INSERT
  - NULL 입력
    - INTO절에서 해당 컬럼 이름 생략
    - VALUES절에서 **NULL** 키워드 사용
  - 기본값 처리
    - INTO절에서 해당 컬럼 이름 생략
    - VALUES절에서 **DEFAULT** 키워드 사용

DML

UPDATE

- 기존 데이터를 수정
- 레코드 수 변화 없음

```
UPDATE 테이블명
SET    컬럼명= {값 | Sub Query } [, 컬럼명=값, ...]
[WHERE 조건];
```



- 수정 값은 직접 명시하거나 서브 쿼리의 결과를 이용할 수 있음
- 서브 쿼리의 결과를 이용할 경우 단일 행 서브 쿼리만 가능
- WHERE절 생략 시 전체 레코드가 수정 되니 유의할 것

Error Code: 1175. You are using safe update mode and you tried to update a table without a WHERE that uses a KEY column To disable safe mode, toggle the option in Preferences -> SQL Editor and reconnect.
MySQL의 경우 default가 update와 delete에 대하여 안전모드로 설정이 되어 있다.

1) set SQL_SAFE_UPDATES = 0; (현재 세션에만 적용 : select @@sql_safe_updates)
2) 메뉴의 edit >> preferences >> SQL Editor >> Safe Updates (....)의 check 해제.

UPDATE와 JOIN

- 수정에 쓰일 데이터를 조인된 결과에서 가져와서 처리

```
UPDATE 테이블명 a
JOIN      테이블명 b       ON ( 조인 조건 )
SET       a.컬럼명= b.컬럼명 [, a.컬럼명=b.컬럼명, ...]
[WHERE 조건];
```

DML

- INSERT OR UPDATE
  - 기존 데이터가 있으면 수정 없으면 삽입으로 처리

```
INSERT INTO 테이블명 [(컬럼명 [, 컬럼명...])]
VALUES (값 [, 값...])
ON DUPLICATE KEY UPDATE  컬럼명=값 [, 컬럼명=값, ...] ;
```

DML

- DELETE
  - 기존 데이터 삭제
  - 레코드 수 감소

```
DELETE FROM 테이블명
[WHERE 조건];
```

  - WHERE절 생략 시 전체 레코드가 삭제 되니 유의할 것

SAMSUNG
SW
AI ACADEMY
FOR
YOUTH

함께가요 미래로!
Enabling People

다음
방송에서
만나요!

삼성청년 SW·AI아카데미